Course Recommendation System Documentation

Table of Contents

1. Overview
2. Architecture
3. Model Details
4. Features
5. Installation & Setup
6. Usage Guide
7. Performance Metrics
8. Technical Implementation
9. Troubleshooting
10. Model Limitations

Overview

The Course Recommendation System is a hybrid machine learning solution designed to recommend undergraduate courses to students based on their academic performance, interests, and aptitude scores. The system uses LightGBM (Learning to Rank) with sophisticated feature engineering to address educational ranking challenges.

Key Features

- **Hybrid Approach**: Combines performance-based and interest-based recommendations
- **Subject-Course Alignment**: Direct matching between student strengths and course requirements
- **Stream-Aware Filtering**: Respects academic stream preferences (Science, Commerce, Arts)
- **Penalty System**: Prevents recommendations of courses where students have low subject performance
- **Enhanced Feature Engineering**: 55+ features capturing various student-course relationships

Problem Solved

The system specifically addresses the "Education Problem" where students with poor performance in education-related subjects were still being recommended B.A. Education courses, leading to inappropriate suggestions.

Architecture

```
Input: Student Profile
    ↓
Feature Engineering (55+ features)
    ↓
LightGBM Ranker (Learning-to-Rank)
    ↓
Subject-Course Penalties/Bonuses
    ↓
Stream Filtering
    ↓
Output: Top-K Course Recommendations
```

Model Details

Algorithm: LightGBM Ranker

- **Objective**: LambdaRank (Learning to Rank)
- **Metric**: NDCG@3 (Normalized Discounted Cumulative Gain)
- **Trees**: 500 estimators with early stopping
- **Learning Rate**: 0.06

Model Parameters

python

```python
lgb.LGBMRanker(
    objective="lambdarank",
    metric="ndcg",
    num_leaves=63,
    learning_rate=0.06,
    n_estimators=500,
    feature_fraction=0.85,
    bagging_fraction=0.85,
    bagging_freq=7,
    reg_alpha=0.1,
    reg_lambda=0.1,
    random_state=42,
    importance_type='gain'
)
```

Features

Input Features (35 base features)

Academic Subjects (16):

- Core subjects: physics, chemistry, mathematics, biology, computer_science
- Language & Social: english, hindi
- Commerce: accountancy, business_studies, economics, banking
- Humanities: history, political_science, sociology, education, geography

Interest Scores (16):

- Corresponding interest ratings (1-10) for each subject: `interest_physics`, `interest_chemistry`, etc.

Aptitude Scores (4):

- analytical, verbal, creativity, numerical

Demographics:

- gender, stream (Science/Commerce/Arts)

Engineered Features (20+)

Stream Performance Averages:

- `science_avg`: Average of science subject scores
- `commerce_avg`: Average of commerce subject scores
- `arts_avg`: Average of arts subject scores
- `overall_academic_avg`: Overall academic performance

Interest Averages:

- `science_interest_avg`, `commerce_interest_avg`, `arts_interest_avg`
- `overall_interest_avg`: Overall interest level

Alignment Features:

- `stream_course_match`: Binary match between student stream and course stream
- `subject_course_alignment`: **Critical feature** - Direct subject-course matching with penalties
- `aptitude_total`: Sum of all aptitude scores
- `stream_aptitude_match`: Stream-specific aptitude alignment

Performance-Interest Correlations:

- `science_perf_interest_corr`: Science performance × interest correlation
- `commerce_perf_interest_corr`: Commerce performance × interest correlation
- `arts_perf_interest_corr`: Arts performance × interest correlation

Encoded Features:

- `gender_enc`, `student_stream_enc`, `candidate_course_idx`, `course_stream_idx`

Subject-Course Alignment Logic

The most critical feature addresses direct subject-course matching:

```python
subject_course_mapping = {
    'physics': ['physics'],
    'chemistry': ['chemistry'],
    'mathematics': ['mathematics', 'math'],
    'biology': ['biology', 'botany', 'zoology'],
    'computer_science': ['computer science', 'computer'],
    'english': ['english'],
    'accountancy': ['accounting', 'accountancy'],
    'business_studies': ['business'],
    'economics': ['economics'],
    'banking': ['banking'],
    'history': ['history'],
    'political_science': ['political science'],
    'sociology': ['sociology'],
    'education': ['education'],
    'geography': ['geography'],
    'hindi': ['hindi']
}
```

Penalty/Bonus System:

- **High Performance (≥80)**: +0.8 bonus
- **Low Performance (≤30)**: -0.6 penalty
- **Neutral Performance**: Scaled bonus based on score
- **Interest Alignment**: Additional +0.2 weight for interest scores

Installation & Setup

Dependencies

```python
pandas>=1.5.0
numpy>=1.21.0
lightgbm>=3.3.0
scikit-learn>=1.2.0
joblib>=1.2.0
```

Installation

```bash
pip install pandas numpy lightgbm scikit-learn joblib
```

File Structure

```
course_recommendation_system/
├── course_recommender.py      # Main model class
├── train_model.py             # Training script
├── course_recommendation_model.pkl # Trained model
├── FinalData.csv              # Training data
└── test_scripts/
    ├── test_accuracy.py       # Performance testing
    └── test_custom_students.py # Custom student testing
```

Usage Guide

1. Training a New Model

```python
```

```python
from course_recommender import CourseRecommendationSystem
import pandas as pd

# Load data
df = pd.read_csv('FinalData.csv')

# Initialize system
recommender = CourseRecommendationSystem(
    use_subject_penalties=True,
    penalty_strength=0.6,
    bonus_strength=0.8
)

# Train model
recommender.train(df, test_size=0.2, random_state=42)

# Save model
recommender.save_model('course_recommendation_model.pkl')
```

2. Loading Pre-trained Model

```python
recommender = CourseRecommendationSystem()
recommender.load_model('course_recommendation_model.pkl')
```

3. Making Predictions

```python
```

```python
# Define student profile
student_profile = {
    "gender": "F",
    "stream": "Science",
    "physics": 85,
    "chemistry": 78,
    "mathematics": 92,
    "biology": 70,
    "computer_science": 88,
    "english": 75,
    "interest_physics": 8,
    "interest_chemistry": 7,
    "interest_mathematics": 9,
    "interest_biology": 5,
    "interest_computer_science": 9,
    "interest_english": 6,
    "analytical": 8,
    "verbal": 6,
    "creativity": 7,
    "numerical": 9
}

# Get recommendations
recommendations = recommender.predict_for_student(
    student_profile,
    top_k=3,
    filter_by_stream=True
)

print(recommendations)
```

Output Example:

```
                Course  Score
0     B.Sc. Mathematics  0.891
1  B.Sc. Computer Science  0.847
2          B.Sc. Physics  0.823
```

4. Feature Importance Analysis

python

```
feature_importance = recommender.get_feature_importance(top_n=15)
```

Top Features by Importance:

1. `subject_course_alignment` - Direct subject-course matching
2. `science_perf_interest_corr` - Performance-interest correlation
3. `economics` - Economics subject score
4. `candidate_course_idx` - Course encoding
5. `arts_perf_interest_corr` - Arts performance-interest correlation

Performance Metrics

Training Performance

- **NDCG@3**: ~96.7% (Excellent)
- **Hit Rate@3**: ~94.2%
- **Precision@3**: ~89.1%

Validation Approach

- **Split Method**: By student_id (prevents data leakage)
- **Test Size**: 20% of students
- **Cross-Validation**: Group-wise ranking validation
- **Early Stopping**: 40 rounds patience

Performance Categories

- **Excellent**: NDCG@3 > 90%
- **Good**: NDCG@3 > 80%
- **Fair**: NDCG@3 > 70%
- **Poor**: NDCG@3 ≤ 70%

Technical Implementation

Data Processing Pipeline

1. **Feature Engineering**:
   - Calculate stream averages
   - Compute interest correlations

- Apply subject-course alignment
- Generate penalty/bonus scores

2. **Encoding**:
   - Label encoding for categorical variables
   - Course and stream indexing
   - Gender encoding
3. **Model Training**:
   - Group-wise ranking optimization
   - LambdaRank objective function
   - NDCG metric optimization
4. **Prediction**:
   - Feature reconstruction for new students
   - Ranking score calculation
   - Top-K selection with filtering

Key Classes and Methods

CourseRecommendationSystem:

- `__init__()`: Initialize with penalty parameters
- `train()`: Train the ranking model
- `predict_for_student()`: Generate recommendations
- `save_model()` / `load_model()`: Model persistence
- `get_feature_importance()`: Feature analysis

Core Methods:

- `create_enhanced_features()`: Feature engineering pipeline
- `calculate_subject_course_alignment()`: Subject matching logic
- `course_prefix()`: Extract course stream from name
- `prepare_features()`: Complete feature preparation

Model Architecture Details

Learning-to-Rank Approach:

- Treats recommendation as a ranking problem

- Optimizes for relative ordering rather than absolute scores
- Groups recommendations by student_id
- Uses pairwise learning with LambdaRank

Feature Categories:

1. **Student Characteristics**: Demographics, academic scores, interests
2. **Course Characteristics**: Course stream, subject requirements
3. **Student-Course Interactions**: Alignment scores, performance correlations
4. **Contextual Features**: Stream matching, aptitude alignment

Troubleshooting

Common Issues

1. KeyError: 'subject not found'

```python
# Solution: Ensure all required subjects are in student profile
for subject in required_subjects:
    if subject not in student_profile:
        student_profile[subject] = 0  # Default value
```

2. Empty Recommendations

```python
# Solution: Disable stream filtering
recommendations = recommender.predict_for_student(
    student_profile,
    filter_by_stream=False
)
```

3. Model Not Found

```python
```

```
# Solution: Check file path and retrain if necessary
import os
if not os.path.exists('course_recommendation_model.pkl'):
    # Retrain model
    recommender.train(df)
    recommender.save_model('course_recommendation_model.pkl')
```

4. Poor Performance on Education Courses The system specifically addresses this through the subject-course alignment feature. Students with low education scores receive penalties for education-related courses.

Performance Debugging

Check Feature Importance:

```
python

importance = recommender.get_feature_importance()
print(importance.head(10))
```

Validate Predictions:

```
python

# Test known good/bad cases
test_cases = [
    {"name": "Strong Math Student", "expected": "B.Sc. Mathematics"},
    {"name": "Weak Education Student", "not_expected": "B.A. Education"}
]
```

Model Limitations

Current Limitations

1. **Data Dependency**: Requires comprehensive student profiles with all subject scores
2. **Course Coverage**: Limited to predefined course catalog
3. **Static Mapping**: Subject-course relationships are hardcoded
4. **No Temporal Factors**: Doesn't consider changing interests over time
5. **Binary Stream Logic**: Assumes strict stream boundaries

Assumptions

1. **Subject Relevance**: Past performance predicts future success
2. **Interest Stability**: Student interests remain consistent
3. **Stream Consistency**: Students prefer courses in their academic stream
4. **Linear Relationships**: Performance and interest correlate linearly
5. **Complete Profiles**: All required features are available

Potential Improvements

1. **Dynamic Mappings**: Learn subject-course relationships from data
2. **Temporal Modeling**: Include time-based interest evolution
3. **Uncertainty Quantification**: Provide confidence intervals
4. **Multi-Objective Optimization**: Balance multiple recommendation criteria
5. **Active Learning**: Incorporate user feedback for model improvement

Known Edge Cases

1. **Career Changers**: Students switching streams may receive suboptimal recommendations
2. **Late Bloomers**: Students improving in subjects may not get proper recognition
3. **Interdisciplinary Courses**: Courses spanning multiple subjects may not align well
4. **Regional Variations**: Course names and requirements may vary by institution

---

Model Version: 1.0 **Last Updated**: September 2025 **Compatibility**: Python 3.7+, LightGBM 3.3+ **License**: Educational Use